

Java String Methods

15 String Methods — Real Applications

Name - S.Trivikram

Roll Number - A25126552251

Java String Method 1: length()

Program: UsernameLength

1. Aim

To demonstrate the Java String method **length()** using a practical username length application.

2. Objective

To understand the purpose and usage of **length()**, write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Username Length** and apply the String method **length()** to process the given string data and display a meaningful result.

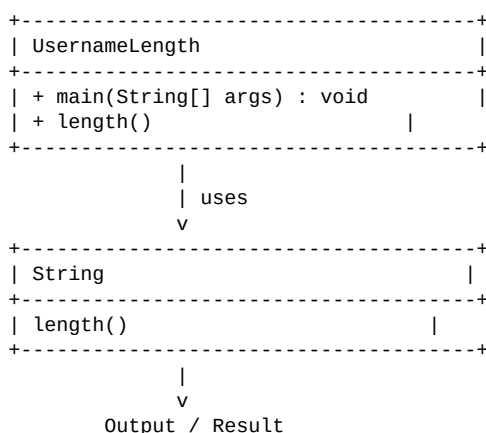
4. Step-wise Execution

1. The program starts execution from the **main()** method.
2. The required String data for username length is created.
3. The String method **length()** is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using **System.out.println()**.

5. Algorithm

1. Start.
2. Create a String variable **username**.
3. Apply **length()** to find the number of characters.
4. Display the username and its length.
5. Stop.

6. UML Class Diagram



7. Program Code

```
class UsernameLength {
    public static void main(String[] args) {
        String username = "Trivikram";
        System.out.println("Username = " + username);
        System.out.println("Username Length = " + username.length());
    }
}
```

8. Code Explanation

class UsernameLength — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

length() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Username = Trivikram  
Username Length = 9
```

10. Result

The program successfully demonstrates the **length()** String method for the real application of username length.

Java String Method 2: toUpperCase()

Program: CityUpperCase

1. Aim

To demonstrate the Java String method **toUpperCase()** using a practical city name application.

2. Objective

To understand the purpose and usage of **toUpperCase()**, write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **City Name** and apply the String method **toUpperCase()** to process the given string data and display a meaningful result.

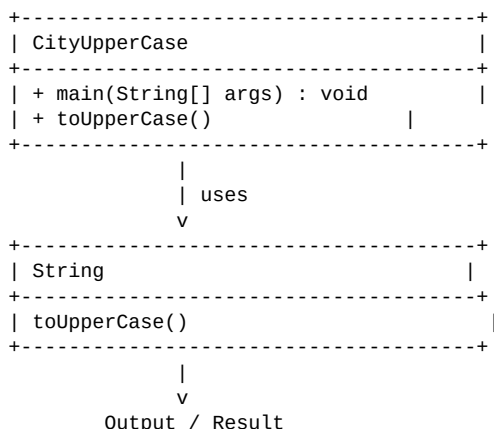
4. Step-wise Execution

1. The program starts execution from the **main()** method.
2. The required String data for city name is created.
3. The String method **toUpperCase()** is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using **System.out.println()**.

5. Algorithm

1. Start.
2. Create a String containing the city name.
3. Apply **toUpperCase()** to convert all letters to uppercase.
4. Display the original and converted city names.
5. Stop.

6. UML Class Diagram



7. Program Code

```
class CityUpperCase {
    public static void main(String[] args) {
        String city = "Visakhapatnam";
        System.out.println("Original City = " + city);
        System.out.println("Uppercase City = " + city.toUpperCase());
    }
}
```

8. Code Explanation

class CityUpperCase — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

toUpperCase() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Original City = Visakhapatnam  
Uppercase City = VISAKHAPATNAM
```

10. Result

The program successfully demonstrates the **toUpperCase()** String method for the real application of city name.

Java String Method 3: toLowerCase()

Program: EmailLowerCase

1. Aim

To demonstrate the Java String method **toLowerCase()** using a practical email address application.

2. Objective

To understand the purpose and usage of **toLowerCase()**, write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Email Address** and apply the String method **toLowerCase()** to process the given string data and display a meaningful result.

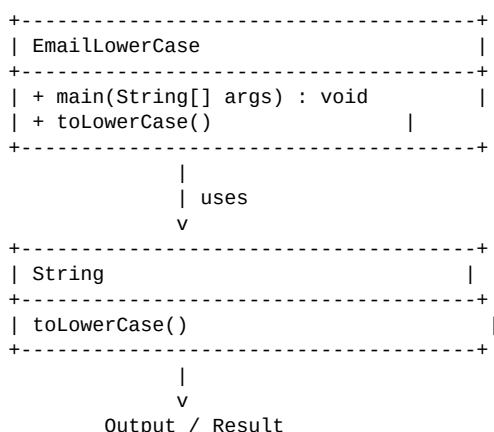
4. Step-wise Execution

1. The program starts execution from the **main()** method.
2. The required String data for email address is created.
3. The String method **toLowerCase()** is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using **System.out.println()**.

5. Algorithm

1. Start.
2. Create a String containing an email address.
3. Apply **toLowerCase()** to convert letters to lowercase.
4. Display the original and converted email.
5. Stop.

6. UML Class Diagram



7. Program Code

```
class EmailLowerCase {
    public static void main(String[] args) {
        String email = "STUDENT@GMAIL.COM";
        System.out.println("Original Email = " + email);
        System.out.println("Lowercase Email = " + email.toLowerCase());
    }
}
```

8. Code Explanation

class EmailLowerCase — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

toLowerCase() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Original Email = STUDENT@GMAIL.COM  
Lowercase Email = student@gmail.com
```

10. Result

The program successfully demonstrates the **toLowerCase()** String method for the real application of email address.

Java String Method 4: charAt()

Program: CustomerInitial

1. Aim

To demonstrate the Java String method **charAt()** using a practical customer initial application.

2. Objective

To understand the purpose and usage of charAt(), write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Customer Initial** and apply the String method **charAt()** to process the given string data and display a meaningful result.

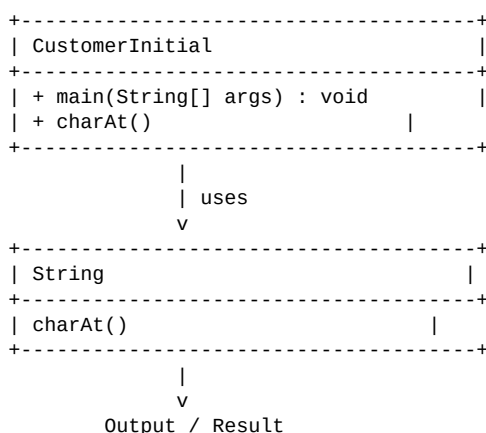
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required String data for customer initial is created.
3. The String method charAt() is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using System.out.println().

5. Algorithm

1. Start.
2. Create a String containing the customer name.
3. Use charAt(0) to obtain the first character.
4. Display the name and initial.
5. Stop.

6. UML Class Diagram



7. Program Code

```
class CustomerInitial {
    public static void main(String[] args) {
        String name = "Rahul";
        char initial = name.charAt(0);
        System.out.println("Customer Name = " + name);
        System.out.println("Customer Initial = " + initial);
    }
}
```

8. Code Explanation

class CustomerInitial — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

charAt() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Customer Name = Rahul  
Customer Initial = R
```

10. Result

The program successfully demonstrates the **charAt()** String method for the real application of customer initial.

Java String Method 5: equals()

Program: PasswordCheck

1. Aim

To demonstrate the Java String method **equals()** using a practical password verification application.

2. Objective

To understand the purpose and usage of equals(), write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Password Verification** and apply the String method **equals()** to process the given string data and display a meaningful result.

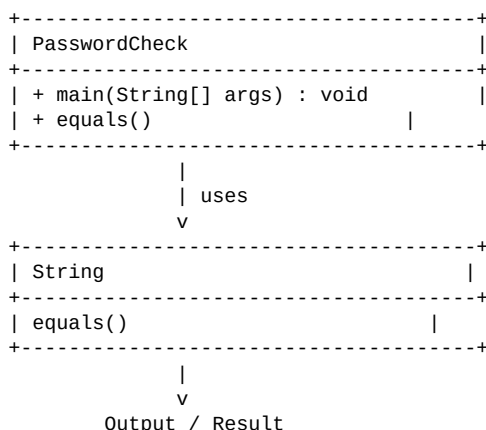
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required String data for password verification is created.
3. The String method equals() is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using System.out.println().

5. Algorithm

1. Start.
2. Store the registered password.
3. Store the password entered by the user.
4. Compare them using equals().
5. If equal, display Password Matched; otherwise display Incorrect Password.
6. Stop.

6. UML Class Diagram



7. Program Code

```
class PasswordCheck {
    public static void main(String[] args) {
        String password = "Java123";
        String enteredPassword = "Java123";
        if (password.equals(enteredPassword)) {
            System.out.println("Password Matched");
        } else {
            System.out.println("Incorrect Password");
        }
    }
}
```

```
        System.out.println("Incorrect Password");
    }
}
```

8. Code Explanation

class PasswordCheck — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

equals() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Password Matched
```

10. Result

The program successfully demonstrates the **equals()** String method for the real application of password verification.

Java String Method 6: equalsIgnoreCase()

Program: UsernameCheck

1. Aim

To demonstrate the Java String method **equalsIgnoreCase()** using a practical username verification application.

2. Objective

To understand the purpose and usage of equalsIgnoreCase(), write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Username Verification** and apply the String method **equalsIgnoreCase()** to process the given string data and display a meaningful result.

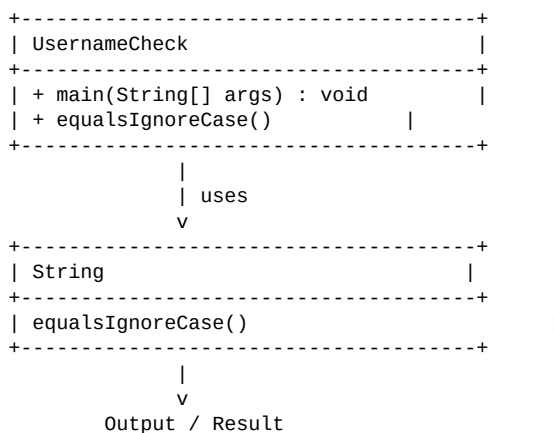
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required String data for username verification is created.
3. The String method equalsIgnoreCase() is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using System.out.println().

5. Algorithm

1. Start.
2. Store the registered username.
3. Store the entered username in a different case.
4. Compare using equalsIgnoreCase().
5. Display whether the username matched.
6. Stop.

6. UML Class Diagram



7. Program Code

```
class UsernameCheck {
    public static void main(String[] args) {
        String registeredName = "Trivikram";
        String enteredName = "TRIVIKRAM";
        if (registeredName.equalsIgnoreCase(enteredName)) {
            System.out.println("Username Matched");
        }
    }
}
```

```
        } else {  
            System.out.println("Username Not Matched");  
        }  
    }  
}
```

8. Code Explanation

class UsernameCheck — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

equalsIgnoreCase() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Username Matched
```

10. Result

The program successfully demonstrates the **equalsIgnoreCase()** String method for the real application of username verification.

Java String Method 7: substring()

Program: EmailUsername

1. Aim

To demonstrate the Java String method **substring()** using a practical extract username from email application.

2. Objective

To understand the purpose and usage of **substring()**, write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Extract Username from Email** and apply the String method **substring()** to process the given string data and display a meaningful result.

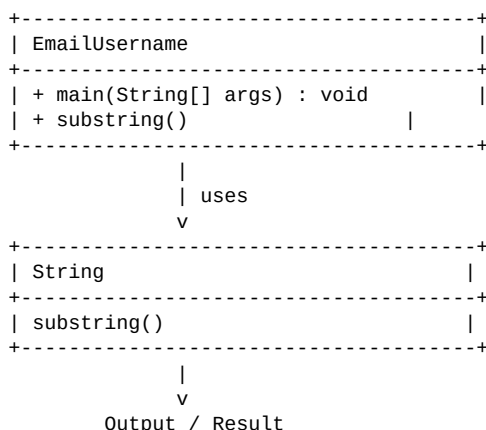
4. Step-wise Execution

1. The program starts execution from the **main()** method.
2. The required String data for extract username from email is created.
3. The String method **substring()** is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using **System.out.println()**.

5. Algorithm

1. Start.
2. Create an email String.
3. Find the **@** position using **indexOf()**.
4. Extract characters before **@** using **substring()**.
5. Display the email and extracted username.
6. Stop.

6. UML Class Diagram



7. Program Code

```
class EmailUsername {
    public static void main(String[] args) {
        String email = "student@gmail.com";
        int position = email.indexOf("@");
        String username = email.substring(0, position);
        System.out.println("Email = " + email);
        System.out.println("Username = " + username);
    }
}
```

```
}  
}
```

8. Code Explanation

class EmailUsername — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

substring() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Email = student@gmail.com  
Username = student
```

10. Result

The program successfully demonstrates the **substring()** String method for the real application of extract username from email.

Java String Method 8: contains()

Program: ProductSearch

1. Aim

To demonstrate the Java String method **contains()** using a practical product search application.

2. Objective

To understand the purpose and usage of contains(), write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Product Search** and apply the String method **contains()** to process the given string data and display a meaningful result.

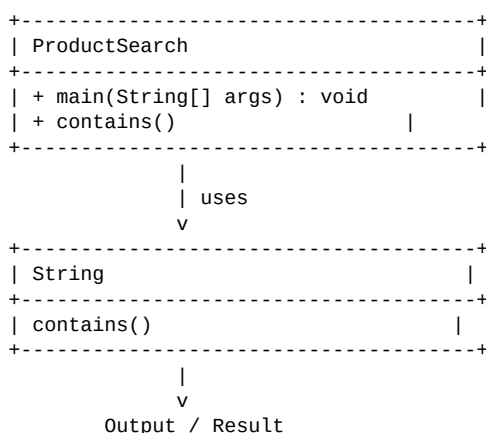
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required String data for product search is created.
3. The String method contains() is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using System.out.println().

5. Algorithm

1. Start.
2. Create a String containing a product description.
3. Use contains() to check for the word Laptop.
4. Display the appropriate search result.
5. Stop.

6. UML Class Diagram



7. Program Code

```
class ProductSearch {
    public static void main(String[] args) {
        String product = "HP Laptop with 8GB RAM";
        if (product.contains("Laptop")) {
            System.out.println("Laptop is available in the product description");
        } else {
            System.out.println("Laptop not found");
        }
    }
}
```

```
}
```

8. Code Explanation

class ProductSearch — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

contains() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Laptop is available in the product description
```

10. Result

The program successfully demonstrates the **contains()** String method for the real application of product search.

Java String Method 9: replace()

Program: ProductURL

1. Aim

To demonstrate the Java String method **replace()** using a practical product url application.

2. Objective

To understand the purpose and usage of **replace()**, write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Product URL** and apply the String method **replace()** to process the given string data and display a meaningful result.

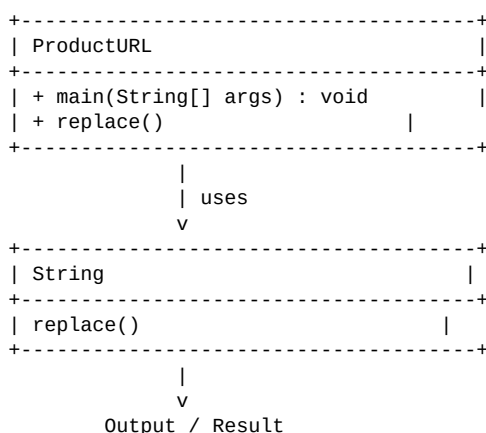
4. Step-wise Execution

1. The program starts execution from the **main()** method.
2. The required String data for product url is created.
3. The String method **replace()** is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using **System.out.println()**.

5. Algorithm

1. Start.
2. Create a product name String.
3. Use **replace()** to replace spaces with hyphens.
4. Display the original and modified names.
5. Stop.

6. UML Class Diagram



7. Program Code

```
class ProductURL {
    public static void main(String[] args) {
        String productName = "Java Programming Book";
        String urlName = productName.replace(" ", "-");
        System.out.println("Product Name = " + productName);
        System.out.println("URL Name = " + urlName);
    }
}
```

8. Code Explanation

class ProductURL — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

replace() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Product Name = Java Programming Book
URL Name = Java-Programming-Book
```

10. Result

The program successfully demonstrates the **replace()** String method for the real application of product url.

Java String Method 10: trim()

Program: CustomerName

1. Aim

To demonstrate the Java String method **trim()** using a practical customer name application.

2. Objective

To understand the purpose and usage of trim(), write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Customer Name** and apply the String method **trim()** to process the given string data and display a meaningful result.

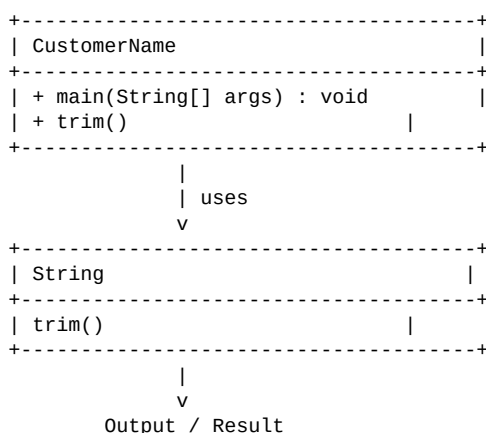
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required String data for customer name is created.
3. The String method trim() is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using System.out.println().

5. Algorithm

1. Start.
2. Create a String containing leading and trailing spaces.
3. Apply trim() to remove the extra spaces.
4. Display the original and cleaned name.
5. Stop.

6. UML Class Diagram



7. Program Code

```
class CustomerName {
    public static void main(String[] args) {
        String name = "  Rahul Kumar  ";
        String cleanName = name.trim();
        System.out.println("Original Name = [" + name + "]");
        System.out.println("Clean Name = [" + cleanName + "]");
    }
}
```

8. Code Explanation

class CustomerName — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

trim() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Original Name = [  Rahul Kumar  ]  
Clean Name = [Rahul Kumar]
```

10. Result

The program successfully demonstrates the **trim()** String method for the real application of customer name.

Java String Method 11: indexOf()

Program: EmailPosition

1. Aim

To demonstrate the Java String method **indexOf()** using a practical email validation application.

2. Objective

To understand the purpose and usage of **indexOf()**, write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Email Validation** and apply the String method **indexOf()** to process the given string data and display a meaningful result.

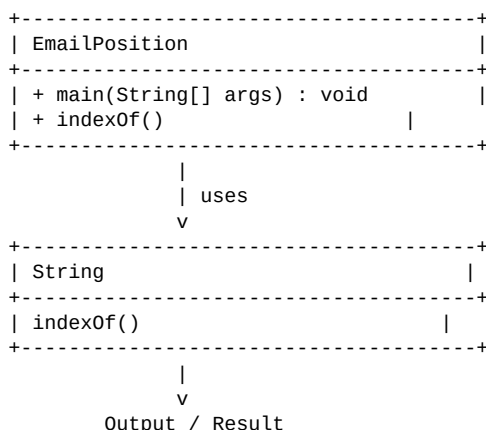
4. Step-wise Execution

1. The program starts execution from the **main()** method.
2. The required String data for email validation is created.
3. The String method **indexOf()** is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using **System.out.println()**.

5. Algorithm

1. Start.
2. Create an email String.
3. Apply **indexOf("@")** to locate the **@** symbol.
4. Store its position.
5. Display the email and position.
6. Stop.

6. UML Class Diagram



7. Program Code

```
class EmailPosition {
    public static void main(String[] args) {
        String email = "student@gmail.com";
        int position = email.indexOf("@");
        System.out.println("Email = " + email);
        System.out.println("@ Position = " + position);
    }
}
```

```
}
```

8. Code Explanation

class EmailPosition — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

indexOf() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Email = student@gmail.com  
@ Position = 7
```

10. Result

The program successfully demonstrates the **indexOf()** String method for the real application of email validation.

Java String Method 12: startsWith()

Program: CountryCode

1. Aim

To demonstrate the Java String method **startsWith()** using a practical country code check application.

2. Objective

To understand the purpose and usage of **startsWith()**, write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Country Code Check** and apply the String method **startsWith()** to process the given string data and display a meaningful result.

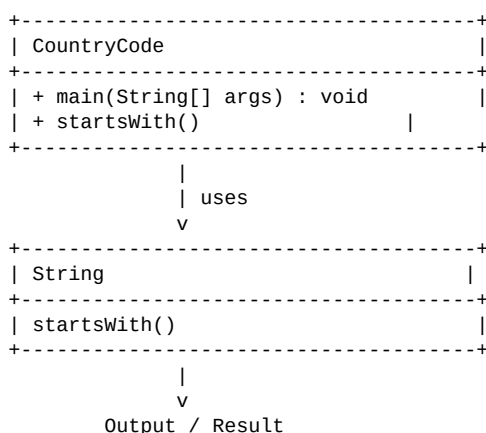
4. Step-wise Execution

1. The program starts execution from the **main()** method.
2. The required String data for country code check is created.
3. The String method **startsWith()** is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using **System.out.println()**.

5. Algorithm

1. Start.
2. Create a phone-number String.
3. Check whether it starts with +91 using **startsWith()**.
4. Display the corresponding country result.
5. Stop.

6. UML Class Diagram



7. Program Code

```
class CountryCode {
    public static void main(String[] args) {
        String phone = "+91-9876543210";
        if (phone.startsWith("+91")) {
            System.out.println("Indian Mobile Number");
        } else {
            System.out.println("Other Country Number");
        }
    }
}
```

```
}
```

8. Code Explanation

class CountryCode — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

startsWith() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Indian Mobile Number
```

10. Result

The program successfully demonstrates the **startsWith()** String method for the real application of country code check.

Java String Method 13: endsWith()

Program: FileCheck

1. Aim

To demonstrate the Java String method **endsWith()** using a practical file type check application.

2. Objective

To understand the purpose and usage of **endsWith()**, write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **File Type Check** and apply the String method **endsWith()** to process the given string data and display a meaningful result.

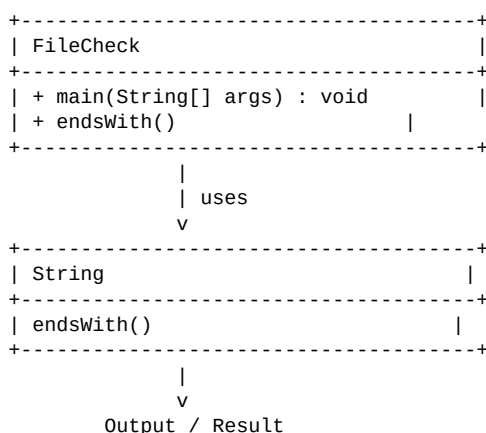
4. Step-wise Execution

1. The program starts execution from the `main()` method.
2. The required String data for file type check is created.
3. The String method `endsWith()` is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using `System.out.println()`.

5. Algorithm

1. Start.
2. Create a file-name String.
3. Check whether it ends with `.pdf` using `endsWith()`.
4. Display the file-type result.
5. Stop.

6. UML Class Diagram



7. Program Code

```
class FileCheck {
    public static void main(String[] args) {
        String fileName = "JavaNotes.pdf";
        if (fileName.endsWith(".pdf")) {
            System.out.println("This is a PDF file");
        } else {
            System.out.println("This is not a PDF file");
        }
    }
}
```

```
}
```

8. Code Explanation

class FileCheck — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

endsWith() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
This is a PDF file
```

10. Result

The program successfully demonstrates the **endsWith()** String method for the real application of file type check.

Java String Method 14: concat()

Program: FullName

1. Aim

To demonstrate the Java String method **concat()** using a practical full name application.

2. Objective

To understand the purpose and usage of concat(), write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Full Name** and apply the String method **concat()** to process the given string data and display a meaningful result.

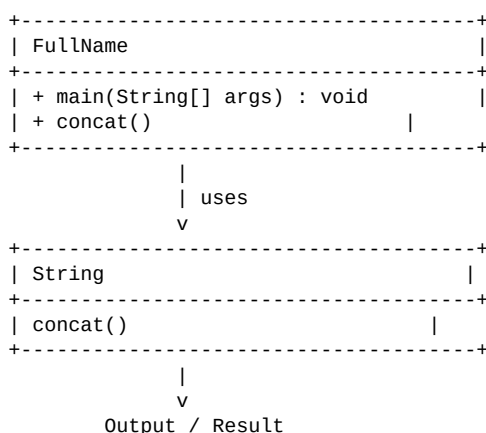
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required String data for full name is created.
3. The String method concat() is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using System.out.println().

5. Algorithm

1. Start.
2. Create first-name and last-name Strings.
3. Use concat() to join them with a space.
4. Display the complete name.
5. Stop.

6. UML Class Diagram



7. Program Code

```
class FullName {
    public static void main(String[] args) {
        String firstName = "Rahul";
        String lastName = "Kumar";
        String fullName = firstName.concat(" ").concat(lastName);
        System.out.println("First Name = " + firstName);
        System.out.println("Last Name = " + lastName);
        System.out.println("Full Name = " + fullName);
    }
}
```

```
}
```

8. Code Explanation

class FullName — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

concat() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
First Name = Rahul  
Last Name = Kumar  
Full Name = Rahul Kumar
```

10. Result

The program successfully demonstrates the **concat()** String method for the real application of full name.

Java String Method 15: isEmpty()

Program: RegistrationCheck

1. Aim

To demonstrate the Java String method **isEmpty()** using a practical registration form application.

2. Objective

To understand the purpose and usage of isEmpty(), write a Java program using it, and observe the expected result.

3. Task Description

Create a Java program for **Registration Form** and apply the String method **isEmpty()** to process the given string data and display a meaningful result.

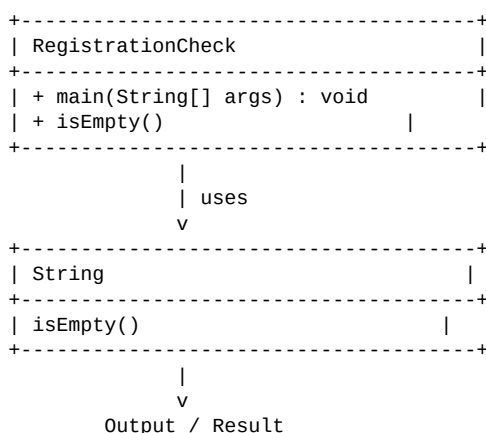
4. Step-wise Execution

1. The program starts execution from the main() method.
2. The required String data for registration form is created.
3. The String method isEmpty() is applied to the data.
4. The returned value or condition is stored/processed as required.
5. The result is displayed using System.out.println().

5. Algorithm

1. Start.
2. Create a String for the registration name.
3. Use isEmpty() to check whether it contains no characters.
4. Display the validation message.
5. Stop.

6. UML Class Diagram



7. Program Code

```
class RegistrationCheck {
    public static void main(String[] args) {
        String name = "";
        if (name.isEmpty()) {
            System.out.println("Name cannot be empty");
        } else {
            System.out.println("Registration Successful");
        }
    }
}
```

```
}
```

8. Code Explanation

class RegistrationCheck — Defines the class containing the application.

public static void main(String[] args) — Starting point of program execution.

String data — Stores the input text required by the application.

isEmpty() — Performs the required String operation.

System.out.println() — Displays the result to the user.

9. Expected Output / Output Screenshot

When the program is executed, the output is:

```
Name cannot be empty
```

10. Result

The program successfully demonstrates the **isEmpty()** String method for the real application of registration form.

Summary of 15 String Methods

No.	String Method	Application
1	length()	Username Length
2	toUpperCase()	City Name
3	toLowerCase()	Email Address
4	charAt()	Customer Initial
5	equals()	Password Verification
6	equalsIgnoreCase()	Username Verification
7	substring()	Extract Username from Email
8	contains()	Product Search
9	replace()	Product URL
10	trim()	Customer Name
11	indexOf()	Email Validation
12	startsWith()	Country Code Check
13	endsWith()	File Type Check
14	concat()	Full Name
15	isEmpty()	Registration Form